

ECM2433 Mock Exam Paper 3

The C Family: C, C++, and Rust

Module: ECM2433

Time allowed: 2 Hours

Total marks: 100

Instructions

- Answer all questions.
- Marks for each part of a question are shown in brackets.
- Total marks for this paper: 100.

1 Question 1: C Programming, Pointer Arithmetic, and Memory Layout (25 marks)

(a)

A programmer defines the following macro to calculate the product of two numbers:

```
#define MULT(a, b) a * b
```

- (i) What is the result of `printf("%d\n", MULT(2 + 3, 4 + 5));`? Show the intermediate expansion step. [4 marks]
- (ii) Provide the corrected macro definition and explain why parentheses are necessary for both the individual arguments and the whole expression. [4 marks]

(b)

In C, arrays are stored in **row-major order**. Given the declaration `int A[3][4];`, calculate the memory offset (in number of integers) from the start of the array to the element `A[2][1]`. Explain how the compiler uses the array dimensions to find this address. [4 marks]

(c)

Write a C function with the following prototype:

```
void reverse_string(char *s);
```

The function must reverse the string in place using only **pointer arithmetic** (no square brackets `[]` and no extra buffers). You may use `strlen()`. [5 marks]

(d)

A programmer writes a “wild pointer” error:

```
1 int *p;  
2 *p = 10;
```

Explain what a **wild pointer** is and contrast it with a **dangling pointer**. How does using `valgrind` help identify these errors? [4 marks]

(e)

Explain the difference between `fprintf(stdout, "Log\n");` and `fprintf(stderr, "Error\n");`. Why is it considered good practice to use `stderr` for error messages even if both appear on the terminal by default? [4 marks]

2 Question 2: C++ Object-Oriented Programming and Templates (25 marks)

(a)

C++ supports **multiple inheritance**. Consider the following base classes:

```
1 class Vehicle { public: int wheels; };
2 class Engine { public: float power; };
```

- (i) Give the code to derive a class `PetrolCar` from both `Vehicle` and `Engine` using **public inheritance**. [2 marks]
- (ii) Write a constructor for `PetrolCar` that uses a **member initialiser list** to pass values to both base class constructors. [4 marks]

(b)

Explain the concept of **Function Overloading** in C++. How does the compiler distinguish between `void print(int i)` and `void print(double d)` at the object code level? [4 marks]

(c)

A developer is using a `std::map<int, Student>` to store student records where the `int` is the student ID.

- (i) Write a code snippet to check if a student with ID 1001 exists in the map. If they exist, print their name. [4 marks]
- (ii) Why is a `std::map` often preferred over a `std::vector` for searching by ID? [3 marks]

(d)

Templates allow for generic programming.

- (i) Define a template function `T find_max(T a, T b)` that returns the larger of two values. [3 marks]
- (ii) What happens if you call `find_max` with two `std::string` objects? Why does this work? [3 marks]

(e)

Explain why a class that contains a `std::unique_ptr` cannot be copied using a standard assignment (e.g., `obj1 = obj2;`). What must the programmer do to transfer the pointer to another object? [2 marks]

3 Question 3: Rust Ownership, Lifetimes, and Concurrency (25 marks)

(a)

Consider this function designed to return the longer of two string slices:

```
1 fn longest(x: &str, y: &str) -> &str {  
2     if x.len() > y.len() { x } else { y }  
3 }
```

(i) This code will not compile. Explain the error regarding **lifetimes**. [3 marks]

(ii) Provide the corrected function signature using a lifetime annotation 'a. [2 marks]

(b)

Rust provides `Arc<T>` and `Mutex<T>` for shared state.

(i) What does `Arc` stand for, and why is it necessary instead of `Rc` when working with multiple threads? [3 marks]

(ii) Explain the process of “poisoning” a `Mutex`. When does this happen? [3 marks]

(c)

Write a Rust code snippet that uses a **closure** with `iter()` and `fold()` to calculate the sum of all squares of numbers in a `Vec<i32>`. [5 marks]

(d)

Compare `String` and `&str` in Rust.

(i) Which one owns its data on the heap? [2 marks]

(ii) Why is it generally better to use `&str` as a function parameter type if the function only needs to read the text? [3 marks]

(e)

What is the difference between `unwrap()` and `expect("message")` when handling an `Option` or `Result`? In what scenario is `expect` considered better practice? [4 marks]

4 Question 4: Language Comparison and Robustness (25 marks)

(a) Memory Safety

- (i) In C, what is a **Segmentation Fault**? Give an example of code that would trigger one. [4 marks]
- (ii) How does Rust's ownership system prevent a "Use After Free" error at compile time, whereas C requires tools like **valgrind** to find it at runtime? [5 marks]

(b) Error Handling

- (i) Contrast the use of **Exceptions** in C++ with the **Result enum** in Rust. Give one advantage of the Rust approach regarding visibility of potential failures. [4 marks]
- (ii) Explain how the `?` operator in Rust handles the propagation of errors compared to manual `if (err)` checks in C. [4 marks]

(c) Abstraction

Compare C++ **Templates** with Rust **Traits**.

- (i) How does a **trait bound** (e.g., `<T: PartialOrd>`) ensure that a generic function is safe to compile? [4 marks]

(d) Resource Management

Describe the **RAII** pattern in C++. How does the destructor of a `std::vector` or `std::unique_ptr` ensure that memory is not leaked even if a function exits via an exception? [4 marks]

End of Paper